Machine Learning 2 (APM_50441_EP/DSAIB - 2026-2027). Lab2

The objectives of the lab

This lab illustrates the second lecture (*A Review of the Methods Seen So Far*). Its goals are:

- to compare classifiers with the *truth*: on a simulated problem, we know the conditional probability $\eta(x) = \mathbb{P}(Y = \text{Class1}|X = x)$, hence the Bayes classifier and its risk $\mathcal{R}^\star$, and we can measure the *excess risk* of each method,
- to see the two points of view of the lecture at work: the probabilistic one (plug-in classifiers) and the optimization one (empirical risk minimization of a convex surrogate),
- to measure the bias–variance trade-off of a local method,
- to learn the `tidymodels` workflow (recipes, workflows, tuning, workflow sets, metrics) on the TwoClass data of the lecture.

The optional *To go further* part, after the core, can be skipped at first.

Use of AI. AI assistants (e.g. the ones built into Colab or VS Code) are allowed, but you must be able to explain every line you use, and answer the *predict before you run* questions before running the code.

Core (about 2 hours)

Exercise 1 (The problem and its Bayes classifier)

We simulate a two-class problem in which each class is Gaussian, with different covariances: $X|Y = \text{Class1} \sim \mathcal{N}(\mu_1, \Sigma_1)$ and $X|Y = \text{Class2} \sim \mathcal{N}(\mu_2, \Sigma_2)$, with equal priors. As the model is known, $\eta(x)$ can be computed with Bayes' formula. The following helpers are given:

- `simulate(n, noise = 0)` draws n observations (with `noise` additional useless features),
- `eta(data)` computes $\eta(x)$ for each row,
- `excess_risk(pred_class, test)` computes the excess risk $\mathcal{R}(\hat{f}) - \mathcal{R}^\star = \mathbb{E}\left[\mathbf{1}_{\hat{f}(X) \neq f^\star(X)} |2\eta(X) - 1|\right]$ on a test set where `eta` is known (this formula has a much smaller variance than the difference of two empirical risks),
- `decision_map(fitted, data)` plots the estimated probability of `Class1`, the decision boundary of a fitted model (solid) and the Bayes boundary (dashed).

◦ Plot `train` with the Bayes boundary $\eta(x) = 1/2$, and estimate the Bayes risk $\mathcal{R}^\star = \mathbb{E}[\min(\eta(X), 1 - \eta(X))]$ with `test`. *Predict before you run:* what is the shape of the Bayes boundary?

Exercise 2 (Plug-in classifiers and the estimation of η)

In the probabilistic point of view, we estimate η and plug the estimate in the Bayes rule: $\hat{f}(x) = \text{Class1}$ if $\hat{\eta}(x) > 1/2$. In `tidymodels`, a model is described by a *specification* (`parsnip`): its type, its mode and the *engine* (the R package) that fits it. All the specifications share the same `fit()` and `predict()` interface, and `predict(type = "prob")` gives the estimated probabilities.

1. Fit the six methods on `train`, and plot their decision maps. Which methods can represent the Bayes boundary?

   PC2 (Exercise 1) shows that the excess risk of a plug-in classifier is controlled by the quality of $\hat{\eta}$:

$$\mathcal{R}(\hat{f}) - \mathcal{R}^\star \leq 2\,\mathbb{E}\left[|\hat{\eta}(X) - \eta(X)|\right] \leq \left(2\,\mathbb{E}\left[\mathsf{KL}(\eta(X), \hat{\eta}(X))\right]\right)^{1/2}.$$

2. For each method and for $n \in \{50, 200, 1000, 5000\}$ (with 3 repetitions), compute the excess risk and the two bounds on `test`. Does the bound hold? Is it tight? Which methods have an excess risk that does not vanish when n grows, and why? (The probabilities are clipped in $[10^{-3}, 1 - 10^{-3}]$ in the Kullback–Leibler divergence, which is infinite when $\hat{\eta} \in \{0, 1\}$.)

Exercise 3 (**Empirical risk minimization with convex surrogates**)
In the optimization point of view, we look for $f(x) = \beta^\top \phi(x)$ minimizing the empirical risk of a loss $\ell(Y f(X))$ (with labels $Y \in \{-1, +1\}$, $+1$ for `Class1`). The $0/1$ loss $\mathbf{1}_{Y f(X) \leq 0}$ is replaced by a convex *surrogate*. We minimize it directly with `optim()` (with a very small ridge penalty to avoid divergence on separable data).

1. Write a function `erm(loss, phi, data, lambda = 1e-4)` returning the coefficients $\hat{\beta}$, and compute, for the four losses and both feature maps, the excess risk of $\text{sign}(\hat{f})$ on `test` (with `train`).

2. PC2 (Exercise 2) shows that the minimizer of the surrogate risk has the sign of $2\eta - 1$, and computes the minimizer $\tilde{f}(\eta)$ for each loss: $\log \frac{\eta}{1-\eta}$ (logistic), $\frac{1}{2} \log \frac{\eta}{1-\eta}$ (exponential), $\text{sign}(2\eta - 1)$ (hinge) and $2\eta - 1$ (squared). With the quadratic features, plot $\hat{f}(x)$ against $\eta(x)$ on 2000 test points, together with the curves $\tilde{f}(\eta)$. Which losses give an estimate of η? Check also that the logistic ERM (with `lambda = 0`) gives the same coefficients as the maximum likelihood fit of `logistic_reg()`.

3. The $0/1$ loss is not minimized directly. Starting from the logistic solution with the linear features, plot the empirical $0/1$ risk and the empirical logistic risk along the intercept $\beta_0 + t$, for $t \in [-3, 3]$. Why is the $0/1$ empirical risk hard to minimize?

Exercise 4 (**Local averaging and the bias–variance trade-off**)
The k-nearest neighbors estimate $\eta(x)$ by the proportion of `Class1` among the k closest training points. With the known η, we can measure by Monte Carlo the *bias* $\mathbb{E}[\hat{\eta}(x)] - \eta(x)$ and the *variance* $\text{Var}[\hat{\eta}(x)]$ of the estimate at some points x, over many training sets.

1. *Predict before you run:* how should the squared bias and the variance depend on k? For $k \in \{1, 5, 15, 45, 101\}$, draw 50 training sets of size 200 and compute, averaged over 300 evaluation points, the squared bias and the variance of $\hat{\eta}$, as well as the average excess risk (on the first 5000 test points).

   In practice η is unknown and k is chosen by cross validation. With `tidymodels`, a parameter to tune is marked with `tune()`, and `tune_grid()` evaluates a grid of values on resamples (`rsample`).

2. On one training set of size 200, choose k with a 10-fold cross validation of the accuracy (`tune_grid()`), and compare it with the best k of the Monte Carlo experiment. Then add 8 useless noise features (`simulate(200, noise = 8)`): what happens to the excess risk of the 25-nearest neighbors?

Exercise 5 (**The tidymodels workflow on real data: TwoClass**)
We now use the TwoClass data of the lecture (Kuhn and Johnson, *Applied Predictive Modeling*): 208 observations, two predictors and two classes. The truth is unknown: we have to rely on resampling. A complete `tidymodels` analysis has the following steps:

- `rsample`: split the data into a training and a test set (`initial_split`), and resample the training set (`vfold_cv`),
- `recipes`: describe the preprocessing (normalization, feature design…), estimated on the training part of each resample,
- `parsnip`: describe the models, `workflows`: bundle a recipe and a model,
- `tune` and `workflowsets`: evaluate many workflows and their hyperparameters on the resamples,
- `yardstick`: compute the metrics.

1. Define two recipes (normalized predictors, and normalized predictors with polynomials of degree 2) and the following models: logistic regression, k-NN (k tuned), LDA, QDA, naive Bayes, penalized logistic regression (`glmnet`, penalty and mixture tuned), SVM with a radial kernel (`kernlab`, cost and scale

tuned), a one-hidden-layer perceptron (`brulee`, size and penalty tuned) and a classification tree (cost-complexity tuned). Build a `workflow_set()` crossing them (the polynomial recipe only for the logistic and penalized logistic regressions), and evaluate it with `workflow_map("tune_grid", grid = 6)` on `folds` for the accuracy, the ROC AUC and the Brier score. Rank the workflows.

2. The differences between the best workflows are small. Compare the two best ones with their *paired* differences of ROC AUC on the 15 resamples (their best configurations are evaluated on the same folds). Then finalize the best workflow, fit it on the whole training set and evaluate it once on the test set with `last_fit()`. Plot its decision map.

3. Which phenomena of parts 2 to 4 (misspecification, flexibility, bias–variance trade-off) explain the ranking on TwoClass?

To go further

The following blocks are optional and independent: each of them only uses the objects defined in the core part. Their difficulty is marked by stars: ⋆ direct extension, ⋆⋆ new method, ⋆⋆⋆ longer or research-flavored.

Block	Lecture	Stars
Generative versus discriminative learning curves	Generative modeling, logistic	⋆⋆
The optimism of least squares (PC2, Exercise 3)	Risk estimation, linear regression	⋆⋆
Regularization paths	Regularization	⋆
Kernel trick and random features	SVM	⋆⋆
Double descent	Deep learning, overparametrization	⋆⋆⋆
The width of a perceptron	Neural networks	⋆⋆
AutoML in R: racing and stacking	AutoML (Lecture 1)	⋆⋆
AutoML in R: Bayesian optimization over learners with mlr3	AutoML (Lecture 1)	⋆⋆
Error estimation schemes: ESL Figure 7.1	Risk estimation (Lecture 1)	⋆⋆
Local regression: LOESS	Local averaging	⋆
The variability of cross validation	Risk estimation	⋆

Exercise 6 (**Generative versus discriminative learning curves** ⋆⋆)
Ng and Jordan (2001) compared a generative classifier and its discriminative counterpart: when the generative model is right, it converges faster; when it is wrong, it converges to a worse rule, and the discriminative one wins for large n. LDA and the (linear) logistic regression are such a pair: both give a linear rule (PC2, *Linear discriminant analysis is a linear classifier*), LDA by modeling $P(X \mid Y)$ (Gaussian classes with the same covariance), the logistic regression by modeling $P(Y \mid X)$ directly (linear log-odds). We compare them in dimension $d = 20$ on two problems where η is known:

- (a) Gaussian classes with the same covariance: both models are right;
- (b) independent exponential features, $X_j \mid Y = k \sim \mathcal{E}(\lambda_{kj})$: the log-odds $\log \frac{P(Y=2|x)}{P(Y=1|x)} = \sum_j \left(\log \frac{\lambda_{2j}}{\lambda_{1j}} - (\lambda_{2j} - \lambda_{1j})\, x_j \right.$ are linear (the logistic model is right), but the classes are not Gaussian (LDA is wrong).

◦ Simulate both problems, and compute the excess risk of the LDA and of the logistic regression for $n \in \{30, 50, 100, 200, 500, 1000, 5000, 20000\}$ (averaged over 20 repetitions, on a test sample of 20 000 points). Plot the learning curves in log-log scale.

Exercise 7 (The optimism of least squares $\star\star$)

PC2 (Exercise 3) compares the error of the least squares estimator in the fixed design, repeated fixed design and random design settings. For a well specified Gaussian linear model $Y = X^\top \beta + \varepsilon$ with $X \sim \mathcal{N}(0, I_d)$, the expected training error is $\sigma^2(1 - d/n)$, the repeated fixed design error is $\sigma^2(1 + d/n)$ (the optimism is $2\sigma^2 d/n$, as in Mallows' C_p), and the random design error is $\sigma^2(1 + d/(n - d - 1))$.

- For $n = 100$, $\sigma = 1$ and $d \in \{5, 10, 20, 40, 60, 80, 95\}$, estimate the three errors by Monte Carlo (200 repetitions) and compare them with the formulas. What happens when d approaches n?

Exercise 8 (Regularization paths $\star$)

The penalized logistic regression minimizes the negative log-likelihood plus $\lambda\left((1-\alpha)\|\beta\|_2^2/2 + \alpha\|\beta\|_1\right)$: ridge for $\alpha = 0$, lasso for $\alpha = 1$. The *regularization path* shows the coefficients as functions of λ; the lasso sets some of them exactly to 0.

- On TwoClass with polynomial features of degree 5 (normalized), plot the ridge and lasso paths with `glmnet`, choose λ by cross validation with `cv.glmnet` (minimum and *one standard error* rule), and count the non-zero coefficients of the lasso.

Exercise 9 (Kernel trick and random features $\star\star$)

A kernel SVM never computes the features: it only uses the kernel $k(x, x') = \exp(-\gamma\|x - x'\|^2)$ between pairs of points, at a cost which grows like n^2. *Random Fourier features* (Rahimi and Recht, 2007) approximate this kernel with explicit features $z(x) = \sqrt{2/D}\cos(Wx + b)$, with $W \sim \mathcal{N}(0, 2\gamma I)$ and $b \sim \mathcal{U}[0, 2\pi]$: $z(x)^\top z(x') \approx k(x, x')$.

- On our simulated problem ($n = 2000$), compare the excess risk and the fitting time of an RBF SVM (`kernlab`, $\gamma = 0.5$) with a logistic regression on $D \in \{2, 10, 50, 200\}$ random features, with a small ridge penalty (`glmnet`, penalty 10^{-3}, `mixture = 0`; without any penalty, the fit diverges as soon as the features can almost separate the training points). Check also the approximation of the kernel.

Exercise 10 (Double descent $\star\star\star$)

In a linear regression on D random features, the least squares fit interpolates the data as soon as $D \geq n$. Among all the interpolating solutions, the *minimum norm* one (given by the pseudo-inverse) behaves surprisingly well: its test error, which explodes when $D \approx n$ (as the random design error of the previous blocks), decreases again for $D \gg n$. This *double descent* is one of the clues to understand overparametrized neural networks.

- Regress $y = \sin(2\, a^\top x) + \varepsilon$ ($x \sim \mathcal{N}(0, I_{10})$, a fixed random direction a of norm about 1, $\sigma = 0.3$, $n = 100$) on D random ReLU features $\max(0, w^\top x + b)$ ($w \sim \mathcal{N}(0, I/10)$, $b \sim \mathcal{N}(0, 1)$), with the minimum norm least squares solution, for D from 5 to 3000. Plot the test error as a function of D (median of 20 repetitions, log scales).

Exercise 11 (The width of a perceptron $\star\star$)

The universal approximation theorem says that a perceptron with one hidden layer can approximate any continuous function, provided its hidden layer is large enough. In practice, the width interacts with the training (epochs, penalty).

- Fit `mlp()` (`brulee`, 200 epochs, penalty 10^{-3}) with $1, 2, 4, 8, 16, 32$ hidden units on our simulated problem ($n = 500$), and plot the excess risk and the decision maps for 1, 4 and 32 units.

Exercise 12 (AutoML in R: racing and stacking $\star\star$)

An AutoML system automates what we did by hand in Part 5. Two of its ingredients are available in `tidymodels`: *racing* (`finetune`), which stops evaluating the configurations that are clearly worse after a few resamples (a form of successive halving), and *stacking* (`stacks`), which learns a penalized combination of the out-of-fold predictions of many candidates (as AutoGluon does).

- Run the workflow set of Part 5 with `tune_race_anova()` (grid of 10), compare the number of model fits with the grid search, and build a stacked ensemble of all the candidates with `stacks`. Evaluate it on the test set.

Exercise 13 (**AutoML in R: Bayesian optimization over learners with mlr3** $\star\star$)
`mlr3` is another framework for machine learning in R, with a different vocabulary: a *task* holds the data, a *learner* is a model, a *graph* chains preprocessing and learners, a *tuner* searches the hyperparameters. An AutoML system can be built from its pieces: a graph with a *branch* that selects one learner among several, and a search space in which the hyperparameters of each learner only exist when its branch is selected. A Bayesian optimization (`mlr3mbo`) then chooses the learner and its hyperparameters jointly. (The `mlr3automl` package packages this idea with a larger portfolio, but it needs a Redis server to distribute the search.)

- Build a graph that scales the features and branches between `glmnet`, `kknn`, `ranger` and an RBF `svm`, define the joint search space, and tune it with `tnr("mbo")` (30 evaluations, 5-fold CV of the AUC) on the training part of TwoClass. Which learners were tried, which one is selected, and how does its test AUC compare with the best workflow of Part 5?

Exercise 14 (**Error estimation schemes: ESL Figure 7.1** $\star\star$)
Figure 7.1 of *The Elements of Statistical Learning* shows the training error and the test error of the lasso along its path, for many training sets. We reproduce it on a sparse linear model, and compare the estimates of the test error given by V-fold cross validation, the .632 bootstrap and Mallows' C_p (lecture 1).

- For the model $Y = X^\top \beta + \varepsilon$ with $d = 35$ correlated Gaussian features ($\Sigma_{ij} = 0.5^{|i-j|}$), $\beta = (1, \dots, 1, 0, \dots, 0)$ (15 ones), $\sigma = 2.5$ and $n = 50$, plot the training and test errors of the lasso as functions of λ for 20 training sets. Then, for one training set, compare the 10-fold CV, .632 bootstrap (100 draws) and C_p estimates of the test error along the path.

Exercise 15 (**Local regression: LOESS** $\star$)
LOESS fits, around each point, a weighted linear (or quadratic) regression on its nearest neighbors; the fraction of points used (`span`) plays the role of k in the k-NN.

- On $y = \sin(4x) + \varepsilon$ ($\sigma = 0.3$, $n = 100$), plot the LOESS fits for spans 0.1, 0.3 and 0.9, and choose the span by leave-one-out cross validation.

Exercise 16 (**The variability of cross validation** $\star$) ◦ Repeat a 10-fold cross validation of the logistic regression on the training part of TwoClass with 50 different random splits. How variable is the estimated accuracy? Compare with the differences between the workflows of Part 5.

Use of AI. Artificial intelligence tools have been used to revise the content of this lab. The main contributor takes full responsibility for this content.

License. The texts of this lab are under the Creative Commons Attribution-ShareAlike 4.0 license (CC BY-SA 4.0), and its code under the MIT license; the datasets keep their own licenses. Main contributor: E. Le Pennec. Contributors: S. Boucheron, A. Dieuleveut, A.K. Fermin, S. Gadat, S. Gaiffas, B. Goujaud, A. Guilloux, Ch. Keribin, Th. Klein, E. Matzner, J. Read, M. Sangnier, E. Scornet.